Nectrae Spend Control

Technologies in the local demo at http://127.0.0.1:8787, and the ones the product has already named for the real app.

0	Node 22.13+	Solana devnet	Base Sepolia
npm dependencies	runtime of the demo	test network, live	test network, live

This demo is a spending control for AI agents that pay APIs per call in USDC. Each agent is one API and has one cap. Solana and Base are the networks where the server checks that the USDC left the watched wallet, in the declared amount, to the address the API requested. Both networks count toward the same cap. USDC on these test networks has no value.

The list below is what that process uses. It was read from the running server on 12 October 2026 (GET /api/state) and from the source in mvp/. The production list follows PRD v2, dated 11 October 2026. Where the MVP README still names a later item, that source is marked in the table.

The demo, as it runs

On 12 October 2026 the server on port 8787 was up. Solana devnet answered through the public RPC at api.devnet.solana.com (slot 510,361,329). Base Sepolia answered through sepolia.base.org (block 48,031,571, chain id 84532). SQLite held 3 sample agents and 71 payment rows. The page itself is one HTML file: it draws the chart in SVG and calls /api on the same server.

Solana and Base are JSON-RPC clients written in src/solana.mjs and src/base.mjs. package.json declares no dependencies. The panel does not load @solana/web3.js or @solana/spl-token. ed25519 signing lives in the test payer, scripts/pay.mjs, and runs only when that separate script is started with a secret in .env. The panel never holds a private key and never pays an API.

Technology	Layer	Where it is used	State in this demo
Node.js 22.13+	Runtime	Server process on port 8787. Tested on Node 24.	Running. No npm packages.
Node HTTP server	API	node:http. Agent API at /v1. Panel API at /api.	Running.
SQLite	Data	node:sqlite. File data/nectrae.db.	Running. 3 agents, 71 payments.
node:crypto	Access	Bearer token for the agent. Panel token when the host is not localhost.	Running.
HTML, CSS, JavaScript	Panel	One file, public/index.html. English and Portuguese.	The page at :8787.
SVG	Panel	14-day spend chart drawn in the page itself.	On the page.
Manrope, JetBrains Mono	Panel	Google Fonts. Interface and addresses.	On the page.
Solana JSON-RPC	Verification	Devnet. getSlot, getTransaction, getSignaturesForAddress.	Connected. Slot 510,361,329.
USDC (SPL)	Asset	Circle test mint on devnet. See the contract list.	Recognised on devnet.
Base JSON-RPC	Verification	Sepolia. eth_getTransactionReceipt, eth_getLogs, eth_chainId.	Connected. Block 48,031,571. Chain 84532.
USDC (ERC-20)	Asset	Circle test contract on Sepolia. Transfer event.	Recognised on Sepolia.
Base58	Addresses	Implemented in src/keys.mjs.	In the server.
x402	Product rule	The agent checks the cap before its wallet signs a per-call USDC payment.	A model, not a library.

Technology	Layer	Where it is used	State in this demo
Long-running process	Live agent	The panel loop stays in the server. A separate runner can pay test USDC.	The panel does not sign.
node:test	Tests	npm test. Rules, keys, both validators, and the API.	Not part of the panel process.
ed25519	Test payment	scripts/pay.mjs, only with a secret kept in .env.	Outside the panel.

x402 is the payment pattern the rules enforce: the agent asks the server before its own wallet signs. It is not an installed library.

USDC contracts

The demo recognises one USDC contract per network. The production rows are the Circle contracts the PRD names for mainnet. They are not active in this demo.

Network	Contract
Solana devnet	`4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`
Solana mainnet	`EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v`
Base Sepolia	`0x036CbD53842c5426634e7929541eC2318f3dCF7e`
Base mainnet	`0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913`

How a payment is checked

– The agent calls check with the network (solana or base), the amount, the vendor, and the destination the API returned on HTTP 402.

– The rules engine allows the payment, alerts, blocks it, asks for approval, or pauses the agent. The strictest matching rule wins.

– If the decision is allowed, the agent's own wallet signs. The SDK records the Solana signature or the Base transaction hash with confirm.

– The validator reads that chain and marks the row checked, waiting, divergent, or outside the ledger. A transfer that skipped check is still found by the wallet scan and still counts toward the cap.

What the real app keeps

The production app is the same product on networks where USDC has value. The demo's core stays:

– The rules engine, which decides before any signature.

– The agent SDK: check, wait for approval, and confirm.

– One USDC cap per API. Solana outflow and Base outflow share that cap.

– Read-only verification of amount, sending wallet, and recipient.

– Private keys remain with the agent. The server stores public addresses only, and it refuses a pasted secret.

What the real app adds

These items are named in the PRD or in the MVP README. They are not in the demo process.

Technology	Purpose	Source	How it enters
Solana mainnet	Verify USDC that has value.	PRD, leaving the demo	Official Circle USDC. See the contract list.

Technology	Purpose	Source	How it enters
Base mainnet	Verify USDC that has value.	PRD, leaving the demo	Official Circle USDC. Chain 8453.
Paid RPC	Scan wallets without the public rate limit.	.env.example	Helius, Triton, or QuickNode on Solana, and an equivalent provider on Base.
Message signing	Prove a public address belongs to the operator.	PRD RF-16	Phantom on Solana, and the Base wallet. The key stays in the wallet. This does not pay the API.
Email, Slack, webhook	Alerts outside the panel at 50%, 80%, 100%, and on unregistered outflow.	PRD RF-15	The panel already shows these alerts on screen.
HTTPS	Open the panel off the machine.	Known limit in the PRD	Off localhost, the panel token currently travels over HTTP.
Users and roles	Several people, in place of one panel token.	MVP README, phase 2	The PRD still treats the panel token as the access control.
Always-on host	The live-agent loop needs a process that stays up.	MVP README	Fly, Railway, or a VPS. A serverless function that exits after the request cannot hold the loop.
SPL approve, with revocation	Enforce the cap on Solana itself.	MVP README, phase 2	A token approve does not expire on its own. This item is not yet on the PRD v2 roadmap.

Outside this product

PRD v2, dated 11 October 2026, limits verification to USDC that moves on Solana or Base. Model invoices, card charges, and subscriptions stay outside the product, because that spend never appears as a USDC transfer on either chain. The MVP README still mentions an OpenAI-compatible proxy for model spend; the later PRD removes that promise.

The server does not custody funds. When a wallet connection is added, it is there to prove an address by signing a message. That signature does not pay the API and does not replace the panel token.

Sources

– Running server: GET http://127.0.0.1:8787/api/state on 12 October 2026, and the HTML served at /.
– Code: mvp/package.json, mvp/src, mvp/public/index.html, mvp/sdk, mvp/scripts/pay.mjs.
– Product: docs/PRD-v2-Agent-Spend-Control.md, 11 October 2026.
– Later notes that are not yet in that PRD: the phase-2 section of mvp/README.md.